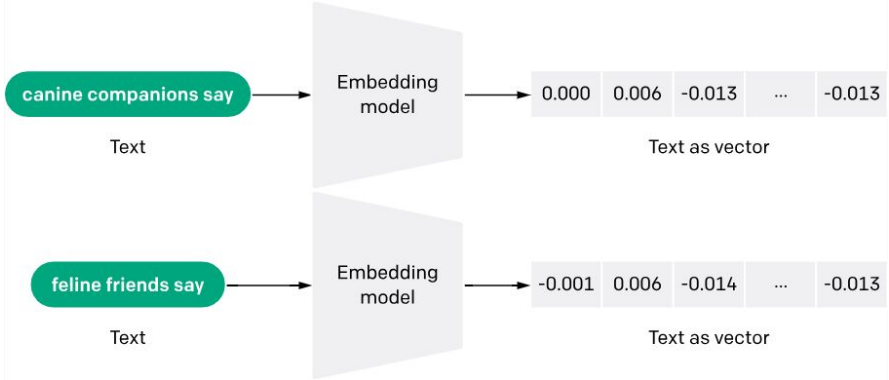
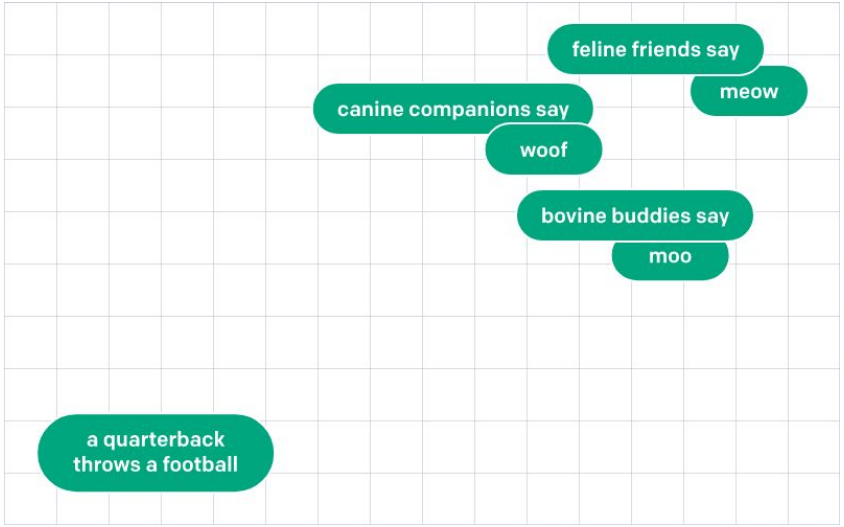
Hierarchical Navigable Small World

Minh Phan

Nearest Neighbor (Vector) Search is important

Neural networks encode information as dense vectors (ex: BERT, ResNet)





Nearest Neighbor Search is important

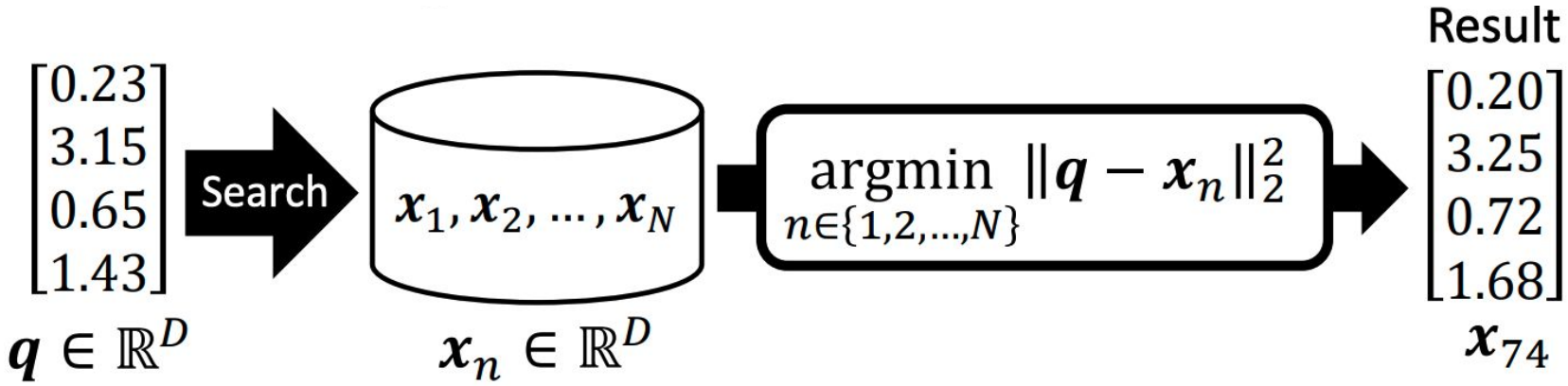
Applications:

- Keyword search
- Recommendations
- RAG systems

Example: Semantic Search

- Query: "Skywalker"
- Finds: "Anakin Skywalker", "Darth Vader", "Luke"

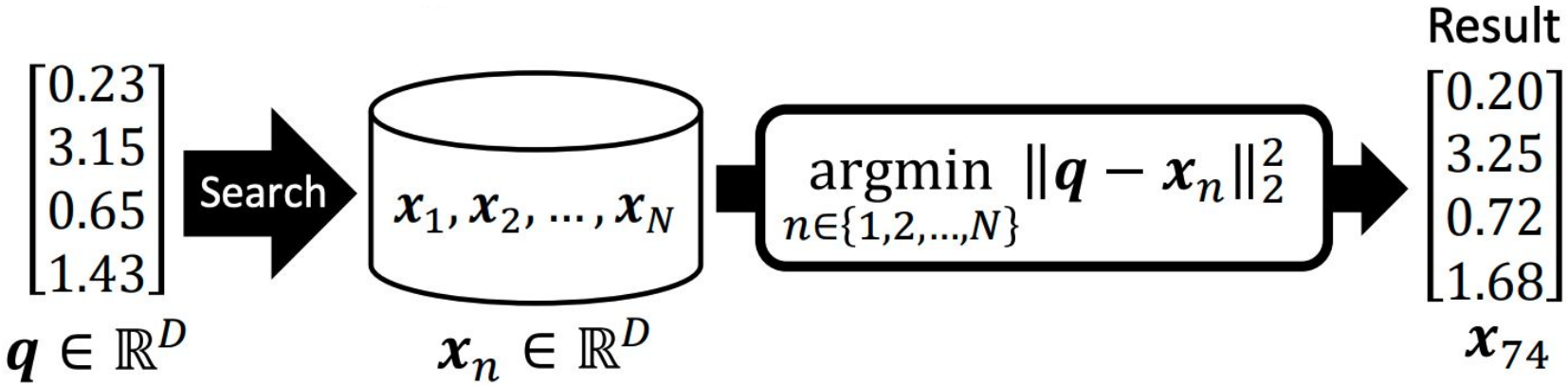
Nearest Neighbor Search



- N vectors with dimension D
- Given a vector query q, find its k nearest neighbors
- Bruteforce: Compare q to N vectors, select k closest → O(N·D) complexity

Yusuke Matsui (CVPR 2020)

Approximate Nearest Neighbor Search

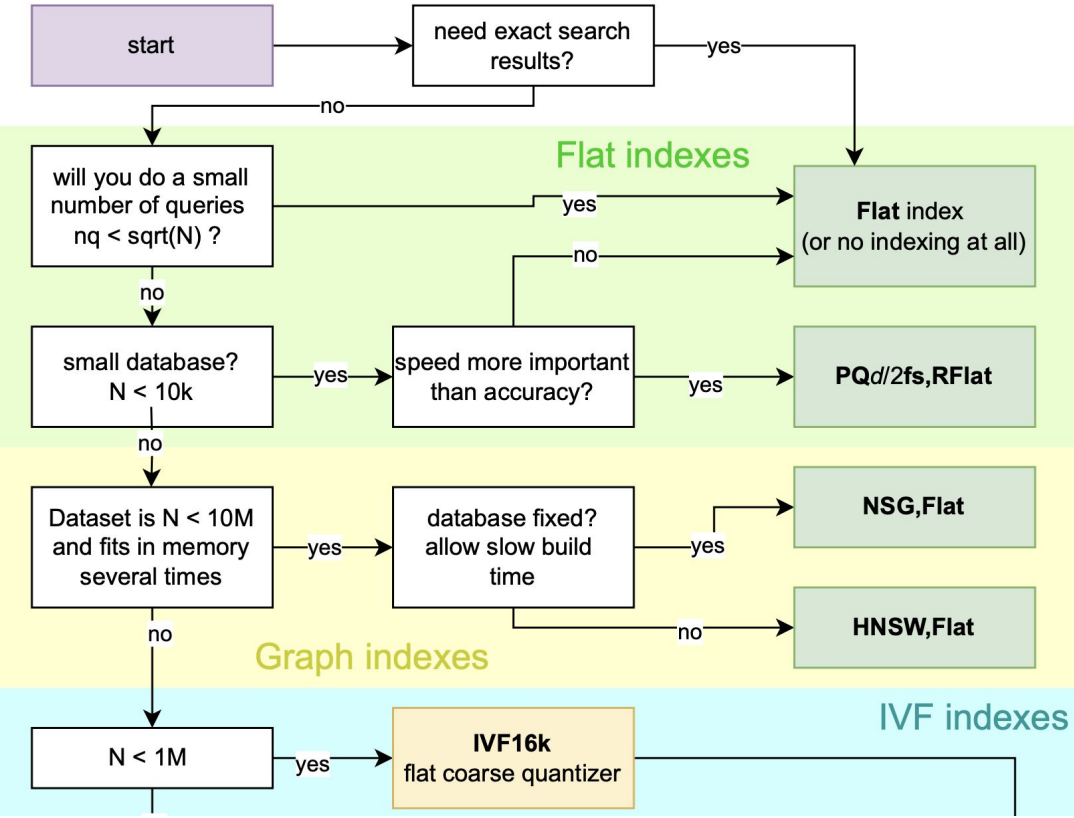


- Return k neighbors that are "close enough" at sub-linear complexity
- Accuracy metrics: recall = # true positive / k
- **Today:** Reduce comparisons using graph indices (HNSW is start-of-the-art)

Yusuke Matsui (CVPR 2020)

Why graph index?

The index that returns most accurate results and has fastest runtime for **medium** size vector sets.



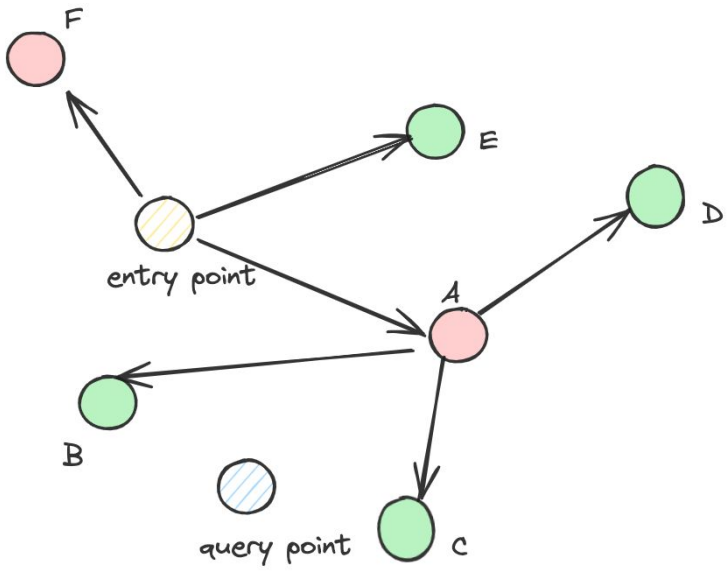
Graph Index

Index Structure

- Nodes are vectors
- Edges define reachability between nodes
- Edge selection strategy varies by algorithm

Key Questions

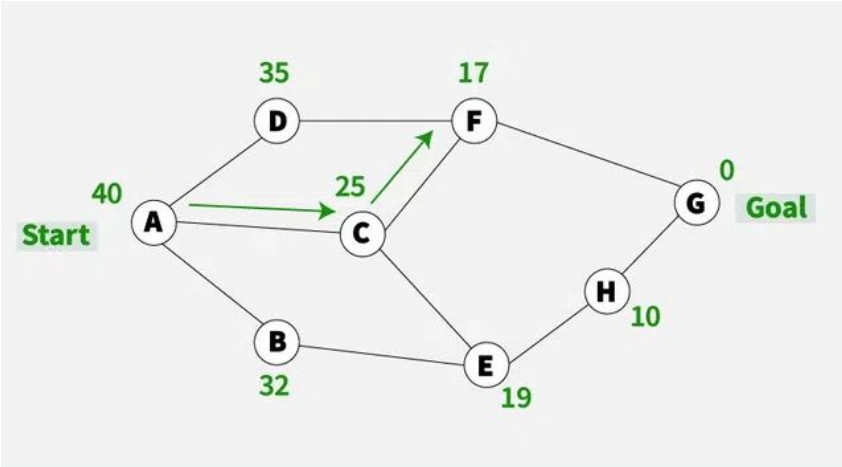
- How to navigate the graph to find nearest neighbors?
- How to select edges that make the graph navigable?
- How do we construct and search the graph efficiently?



Milvus

Greedy Search

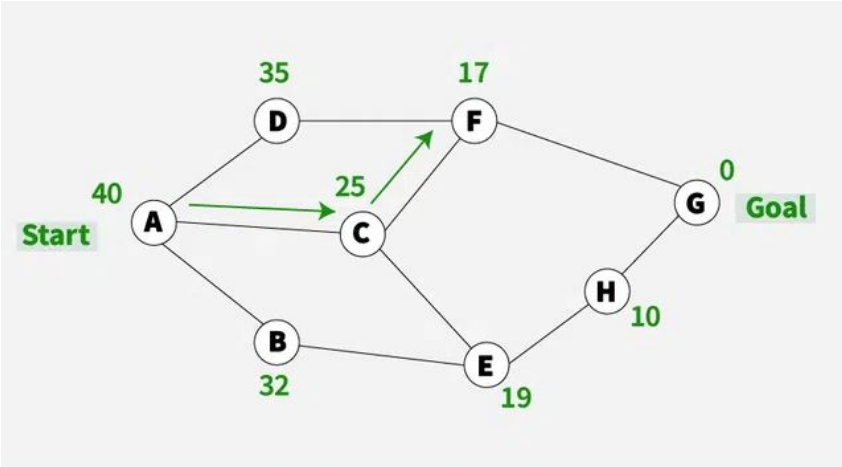
1. Start at an entry point
2. Examine all neighbors of current node
3. Move to the neighbor closest to query
4. Repeat until no neighbor is closer



Beam Search (or Best-first Search)

Parameter: exploration factor (ef)

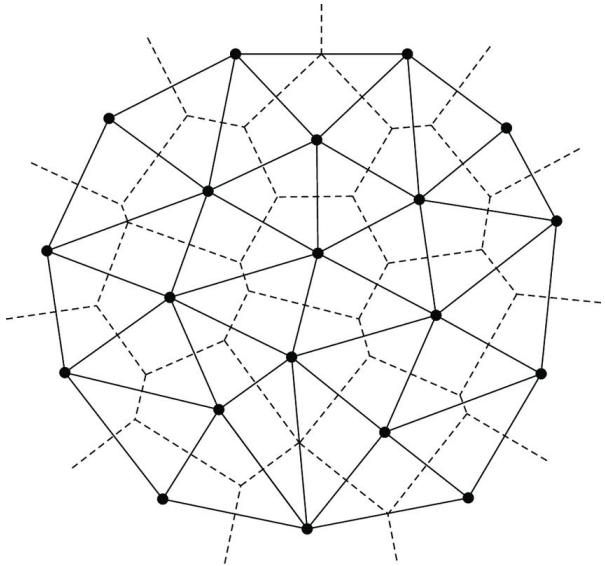
1. Start with entry point
2. Maintain ef best candidates
3. Explore neighbors of ALL candidates
4. Keep track of **ef** nodes closest to query
5. Stop when no new improvements



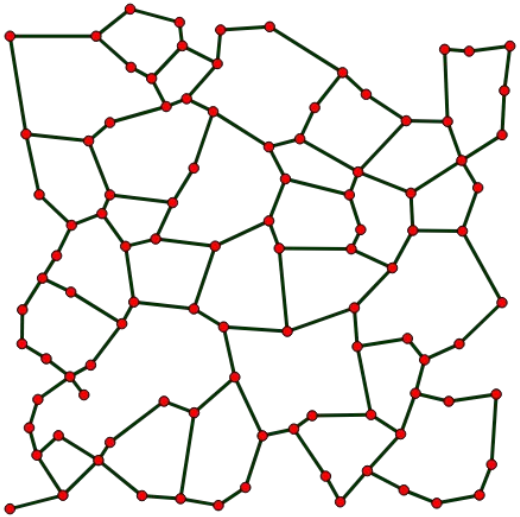
Example: ef = 3

1. Start with node A
2. Explore A's neighbors → keep 3 closest to query [B, C, D]
3. Round 3: Explore B, C, D's neighbors → keep 3 closest [C, E, F]
4. Continue until convergence

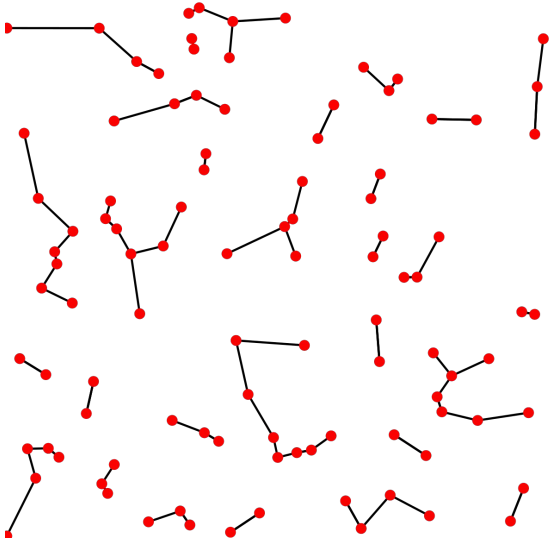
Theoretical models for navigable graph



Delaunay Graph
O(n^⌈d/2⌉)



Relative Neighborhood Graph
O(n^3)



Nearest Neighbor Graph

k-NN Graph

Connect each node to k nearest neighbors

Problems

- Search gets stuck at local minima
- Too many hops needed

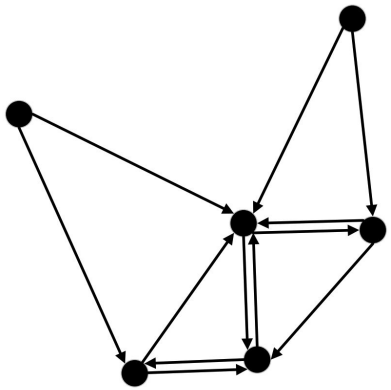


Fig. 2. An illustrative KNNG with $n = 6$ and $k = 2$.

Revisiting k-Nearest Neighbor Graph Construction on High-Dimensional Data : Experiments and Analyses

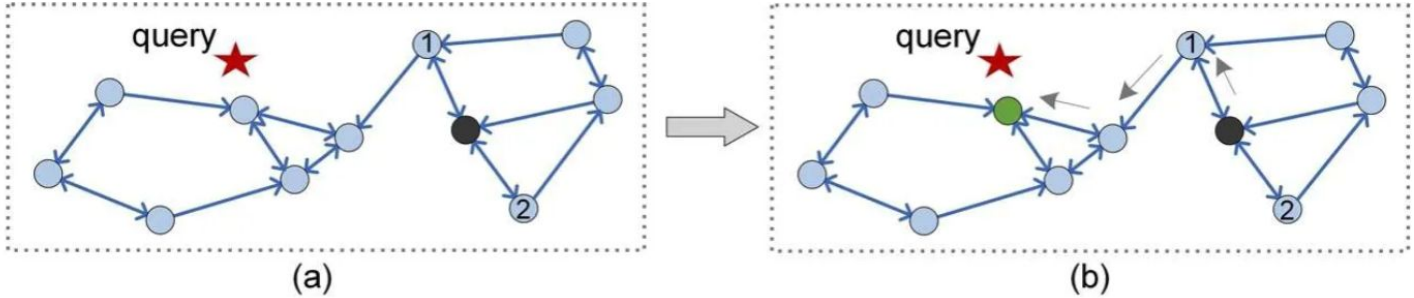


Figure 2: Routing procedure on a PG index (KNNG, k=2).

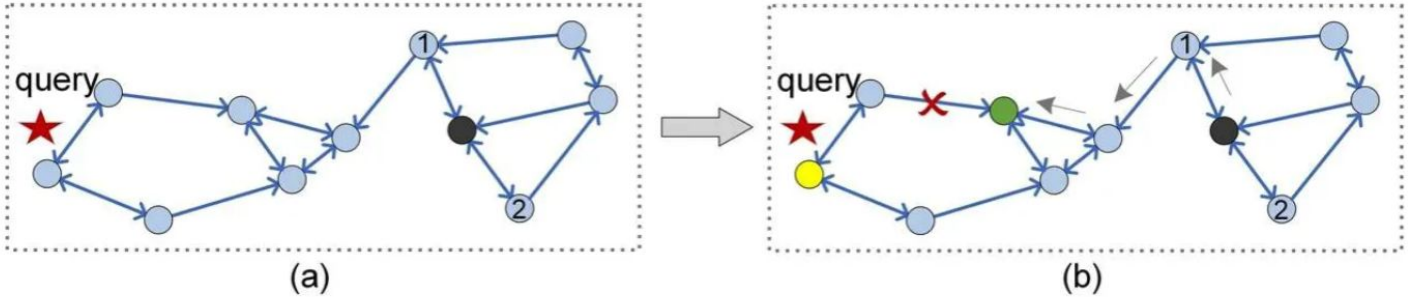
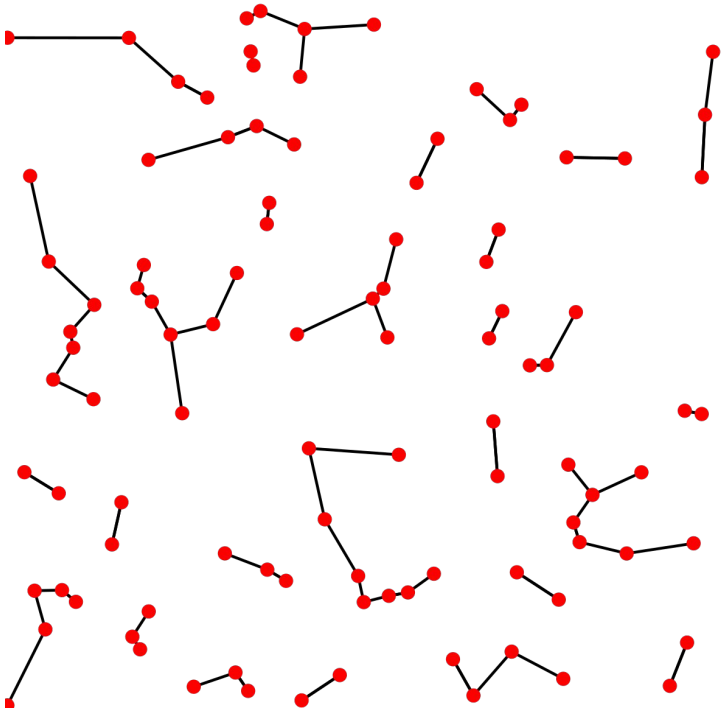
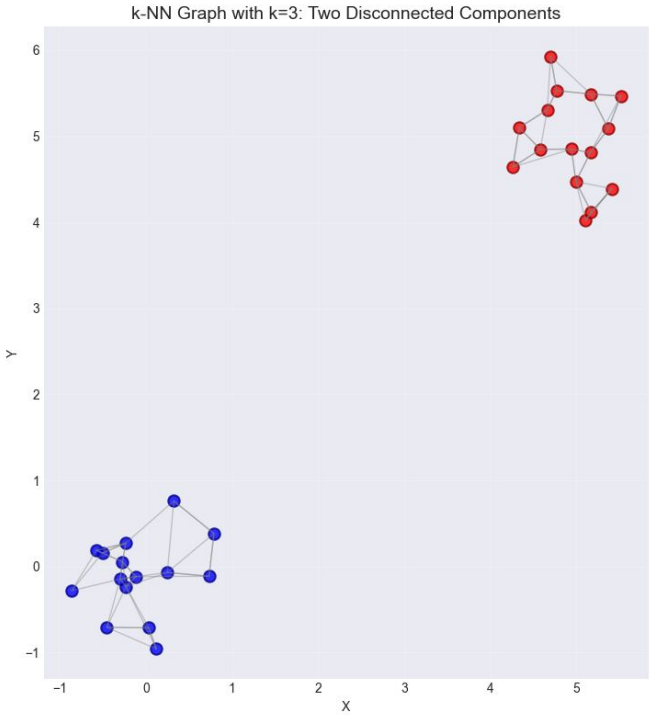


Figure 3: Routing starting from the black vertex cannot reach the yellow vertex.

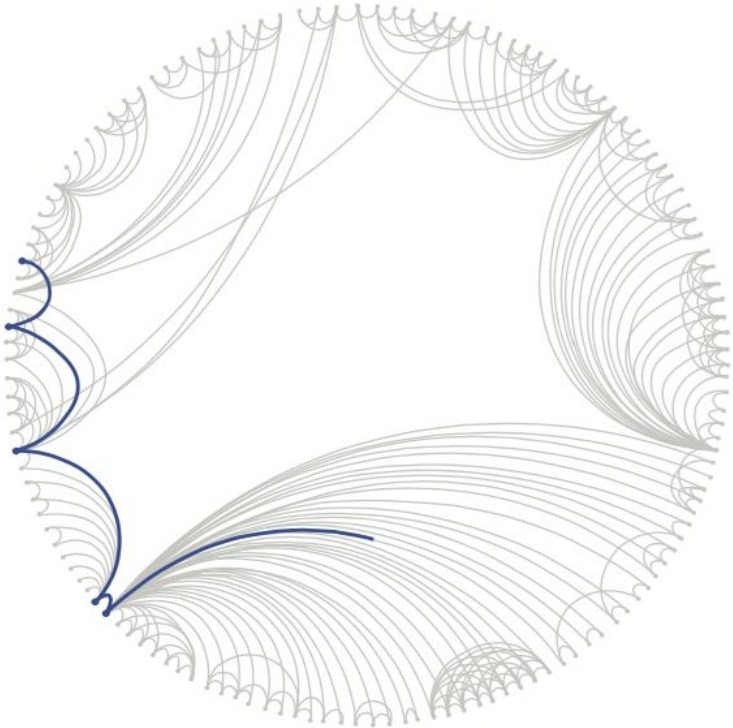
k-NN Graph Limitation





Small World Phenomenon

Milgram's small world experiment (1960s)



Three and a half degree of separation (Facebook 2016)

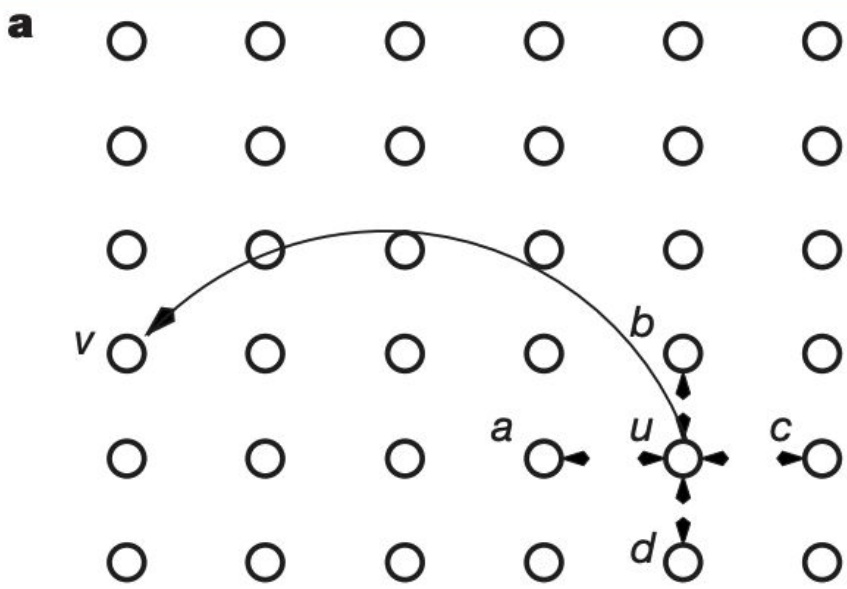
Navigable Small World Graph

Small-world graphs have short paths, but can greedy routing find them?

Kleinberg's Model (2000)

- Start with a 2D grid where each node connects to its neighbors.
- Add long-range edges with probability $P(u,v) \propto 1/\text{distance}(u,v)^2$

➜ Reach any destination in $O(\log^2 n)$ steps instead of $O(\sqrt{n})$ without long edges

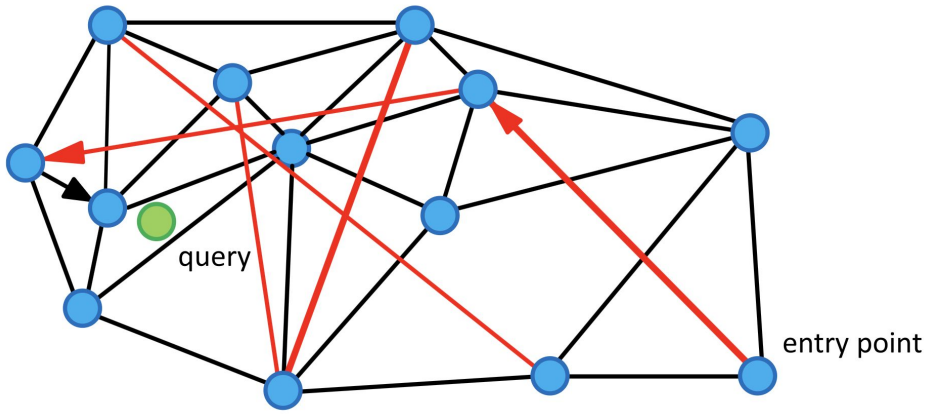


Navigable Small World Graph

Graph structure combining short edges and long edges

Insert nodes incrementally. For each new node:

- Run **m** best-first searches from random existing nodes
- Find **k** nearest neighbors
- Create bidirectional edges



Empirical bound search complexity **O(log^2 N)**

Navigable Small World Graph Pseudocode

```
K-NNSearch(object q, integer: m, k)
 TreeSet [object] tempRes, candidates, visitedSet,
 result
 for (i←0; i<m; i++) do:
   put random entry point in candidates
   tempRes←null
   repeat:
     get element c closest from candidates
     to q
     remove c from candidates
     //check stop condition:
     if c is further than k-th element from result
      than break repeat
      //update list of candidates:
    for every element e from friends of c do:
      if e is not in visitedSet than
        add e to visitedSet, candidates, tempRes

      end repeat
      //aggregate the results:
      add objects from tempRes to result
 end for
 return best k elements from result
```

```
Nearest_Neighbor_Insert(object: new_object, integer:
 f, integer: w)
 SET[object]: neighbors←k-NNSearch (new_object, w,
 f);
 for (I←0; i<f; i++) do
 neighbors [i].connect(new_object);
 new_object.connect(neighbors [i]);
```

Navigable Small World Graph

Problem

- **Polylogarithmic O($\log^2$ N) complexity** - not true logarithmic
- **Random entry points** - must "zoom out" to hubs (nodes with higher degree)
- **Evaluates both long and short edges at each node**
- **Unbounded hub growth** - early nodes accumulate O(log N) edges

Impact

- Poor performance on low-dimensional data
- Unpredictable memory usage
- Inconsistent search times

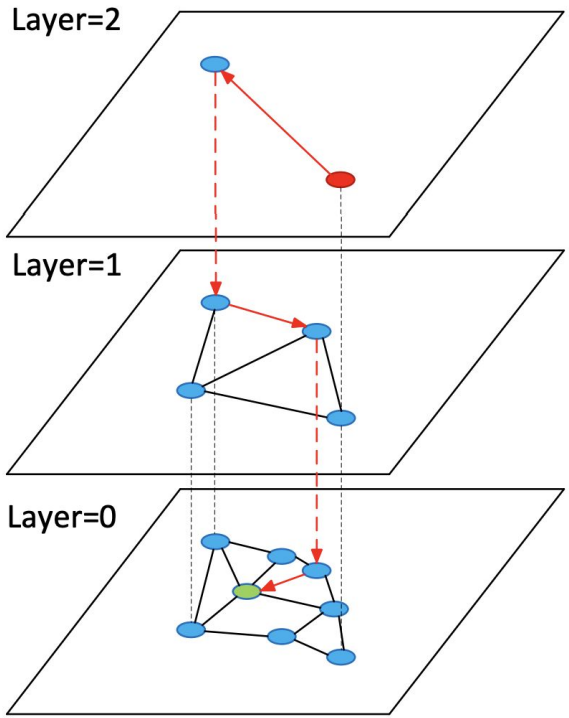
Hierarchical Navigable Small World Graph (HNSW)

Main Idea:

- Add hierarchy to Navigable Small World - separate links by distance scale into layers. At the top, we have the longest links, at the bottom, we have the shortest links.

Result

- Search Complexity: O(log N)
- Construction Complexity: O(NlogN)
- Predictable memory usage



Searching for k nearest neighbor over HNSW

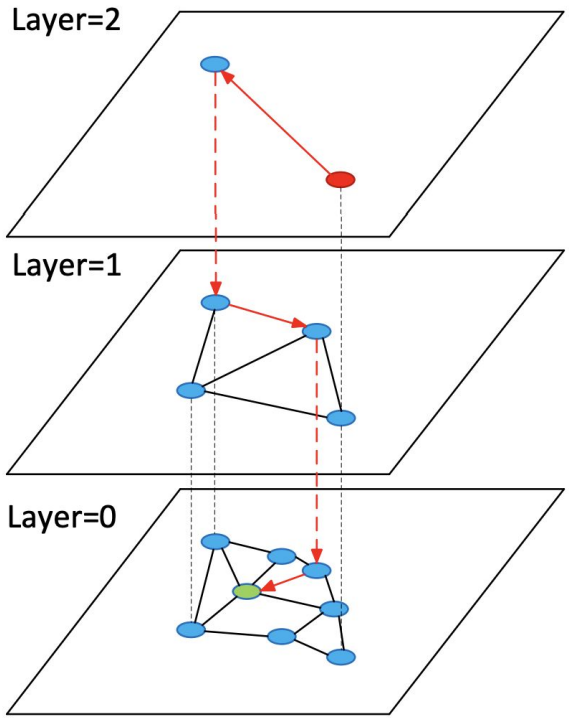
Start: Fixed entry point at top layer

For each layer from top → 1:

- Greedy search to closest node
- This becomes entry point for next layer

Layer 0:

- Best-first search with **ef** candidates
- Return **k** nearest neighbors



Searching for k nearest neighbor over HNSW

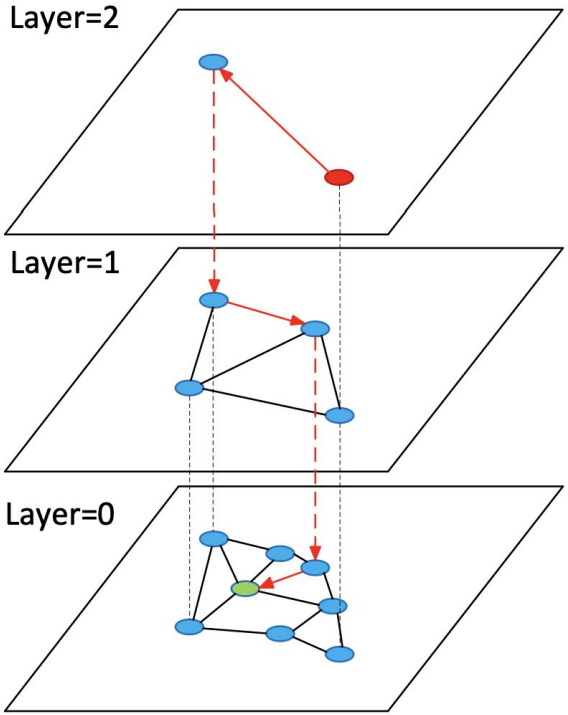
Algorithm 5

K-NN-SEARCH(*hnsw*, *q*, *K*, *ef*)

Input: multilayer graph *hnsw*, query element *q*, number of nearest neighbors to return *K*, size of the dynamic candidate list *ef*

Output: *K* nearest elements to *q*

1 $W \leftarrow \emptyset$ // set for the current nearest elements
2 $ep \leftarrow$ get enter point for *hnsw*
3 $L \leftarrow$ level of *ep* // top layer for *hnsw*
4 **for** $l_c \leftarrow L \ldots 1$
5 $W \leftarrow$ SEARCH-LAYER(q, ep, ef=1, l_c)
6 $ep \leftarrow$ get nearest element from W to q
7 $W \leftarrow$ SEARCH-LAYER(q, ep, ef, l_c =0)
8 **return** K nearest elements from W to q

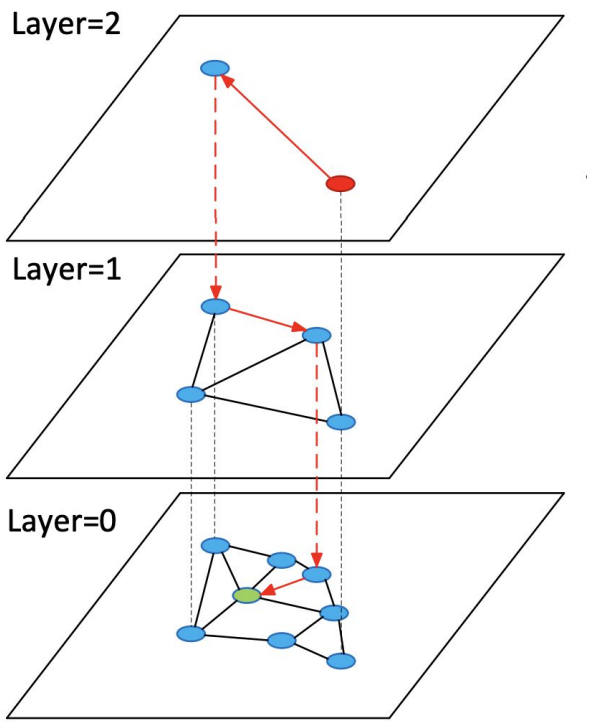


Building the HNSW index

Main Idea

Incrementally insert each node:

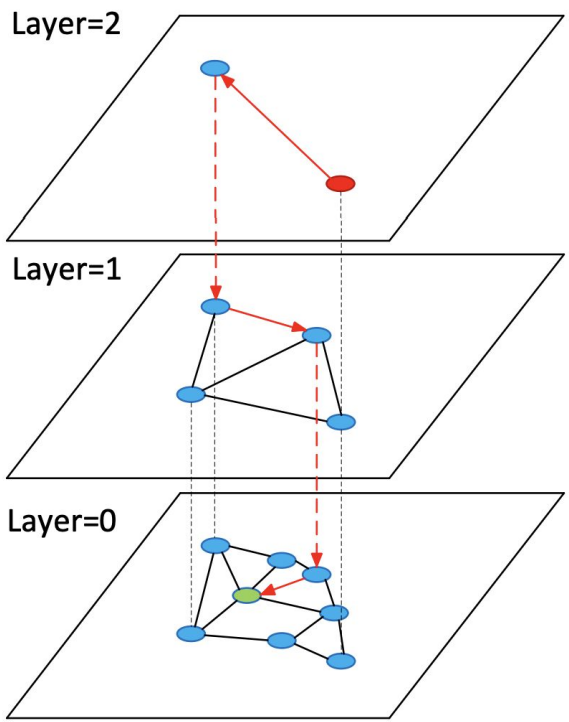
- Select a layer **l** where that node would reside using a random distribution.
- From layer **l** to layer 0, connect n to nearby nodes.
  - In each layer, use the local minima of the previous layer as the entry point.
  - Beam search from that entry point to find M closest nodes.
  - Add connections to those nodes (base on some heuristic)



Building the HNSW index

Algorithm 1
INSERT(*hnsw*, *q*, *M*, M_{max}, *efConstruction*, m_L)
Input: multilayer graph *hnsw*, new element *q*, number of established connections *M*, maximum number of connections for each element per layer M_{max}, size of the dynamic candidate list *efConstruction*, normalization factor for level generation m_L
Output: update *hnsw* inserting element *q*

```
W ← ∅    // list for the currently found nearest elements
ep ← get enter point for hnsw
L ← level of ep    // top layer for hnsw
l ← ⌊-ln(unif(0..1))·mL⌋  // new element's level
for lc ← L … l+1
  W ← SEARCH-LAYER(q, ep, ef=1, lc)
  ep ← get the nearest element from W to q
for lc ← min(L, l) … 0
  W ← SEARCH-LAYER(q, ep, efConstruction, lc)
  neighbors ← SELECT-NEIGHBORS(q, W, M, lc) // alg. 3 or alg. 4
  add bidirectionall connectionts from neighbors to q at layer lc
  for each e ∈ neighbors   // shrink connections if needed
    eConn ← neighbourhood(e) at layer lc
    if |eConn| > Mmax // shrink connections of e
                      // if lc = 0 then Mmax = Mmax0
      eNewConn ← SELECT-NEIGHBORS(e, eConn, Mmax, lc)
                                                // alg. 3 or alg. 4
      set neighbourhood(e) at layer lc to eNewConn
  ep ← W
if l > L
  set enter point for hnsw to q
```

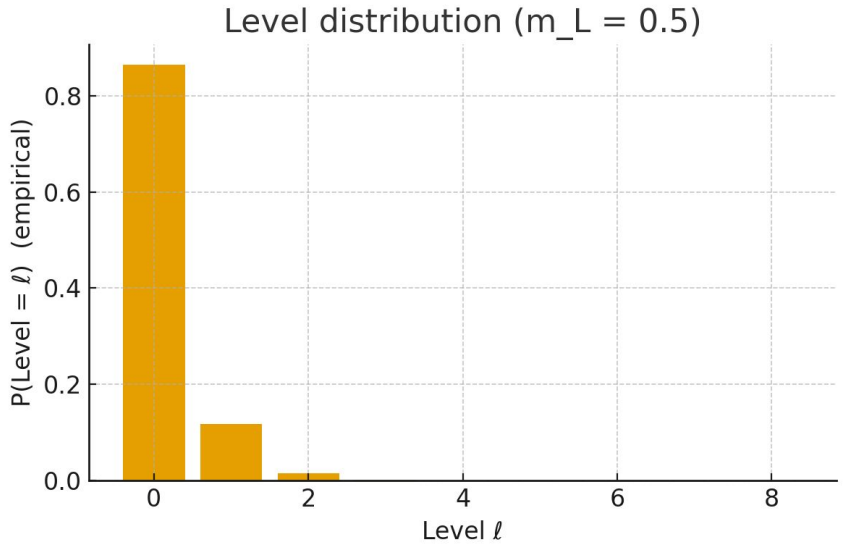


HNSW Layer Assignment

The number of vectors increases exponentially as we move down from higher layers to lower layers.

For example, in practice, an index for **1M** vectors (with appropriate parameters) is likely to have 6 layers.

$$l \leftarrow \lfloor -\ln(unif(0..1)) \cdot m_L \rfloor$$

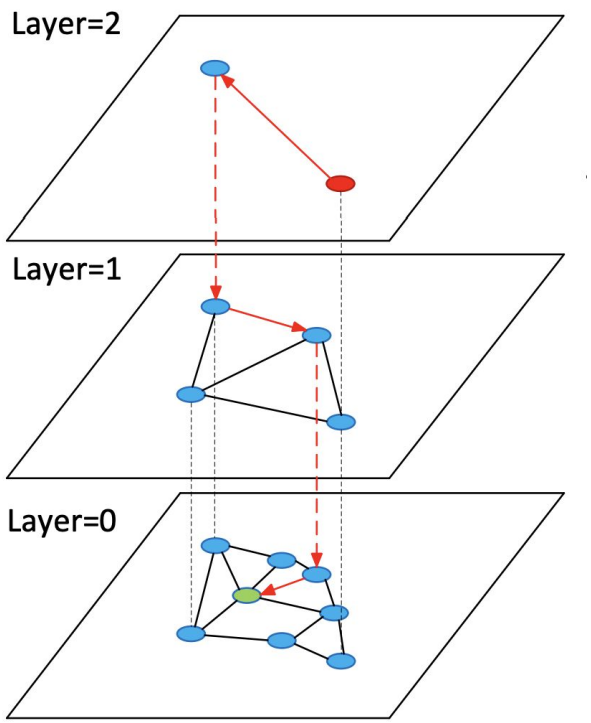


Building the HNSW index

Main Idea

Incrementally insert each node:

- Select a layer **l** where that node would reside using a random distribution.
- From layer **l** to layer 0, connect n to nearby nodes.
  - In each layer, use the **local minima** of the previous layer as the entry point.
  - **Beam search** from that entry point to find M closest nodes.
  - Add connections to those nodes



New Heuristics to select neighbor

Previous Heuristic: Select M closest neighbors, make connection to all those neighbors

Proposed Heuristic:

- For each candidate neighbor C
- If C is closer to new node than to ANY already selected neighbor: Select C
- Else: Skip C

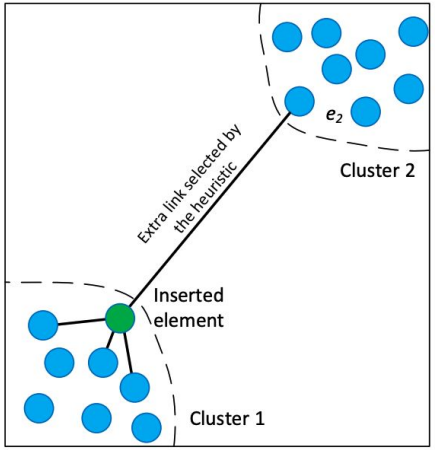


Fig. 2. Illustration of the heuristic used to select the graph neighbors for two isolated clusters. A new element is inserted on the boundary of Cluster 1. All of the closest neighbors of the element belong to the Cluster 1, thus missing the edges of Delaunay graph between the clusters. The heuristic, however, selects element e_2 from Cluster 2, thus, maintaining the global connectivity in case the inserted element is the closest to e_2 compared to any other element from Cluster 1.

New Heuristics to select neighbor

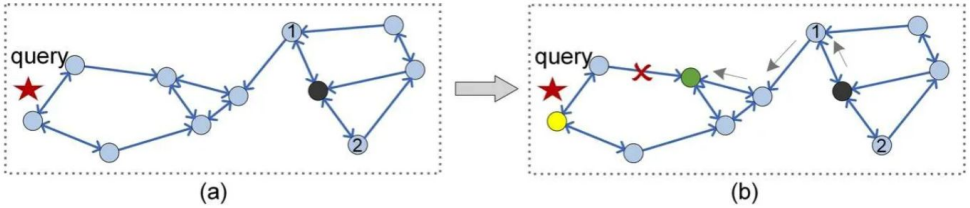


Figure 3: Routing starting from the black vertex cannot reach the yellow vertex.

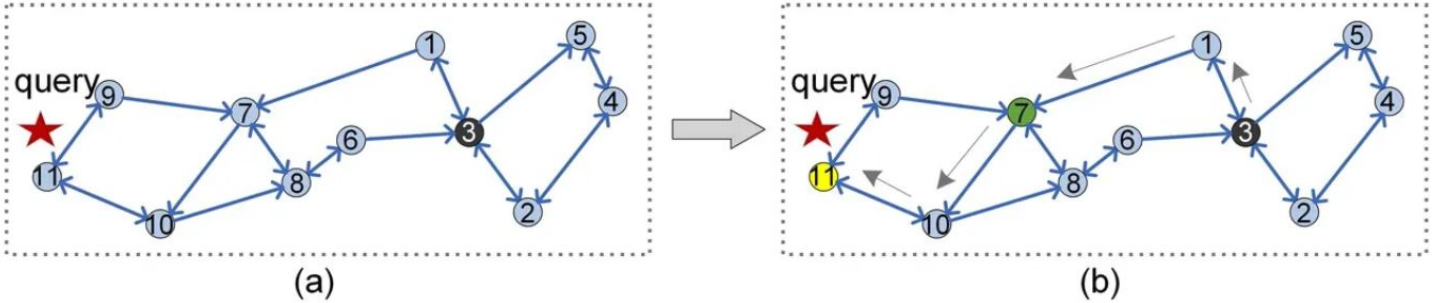
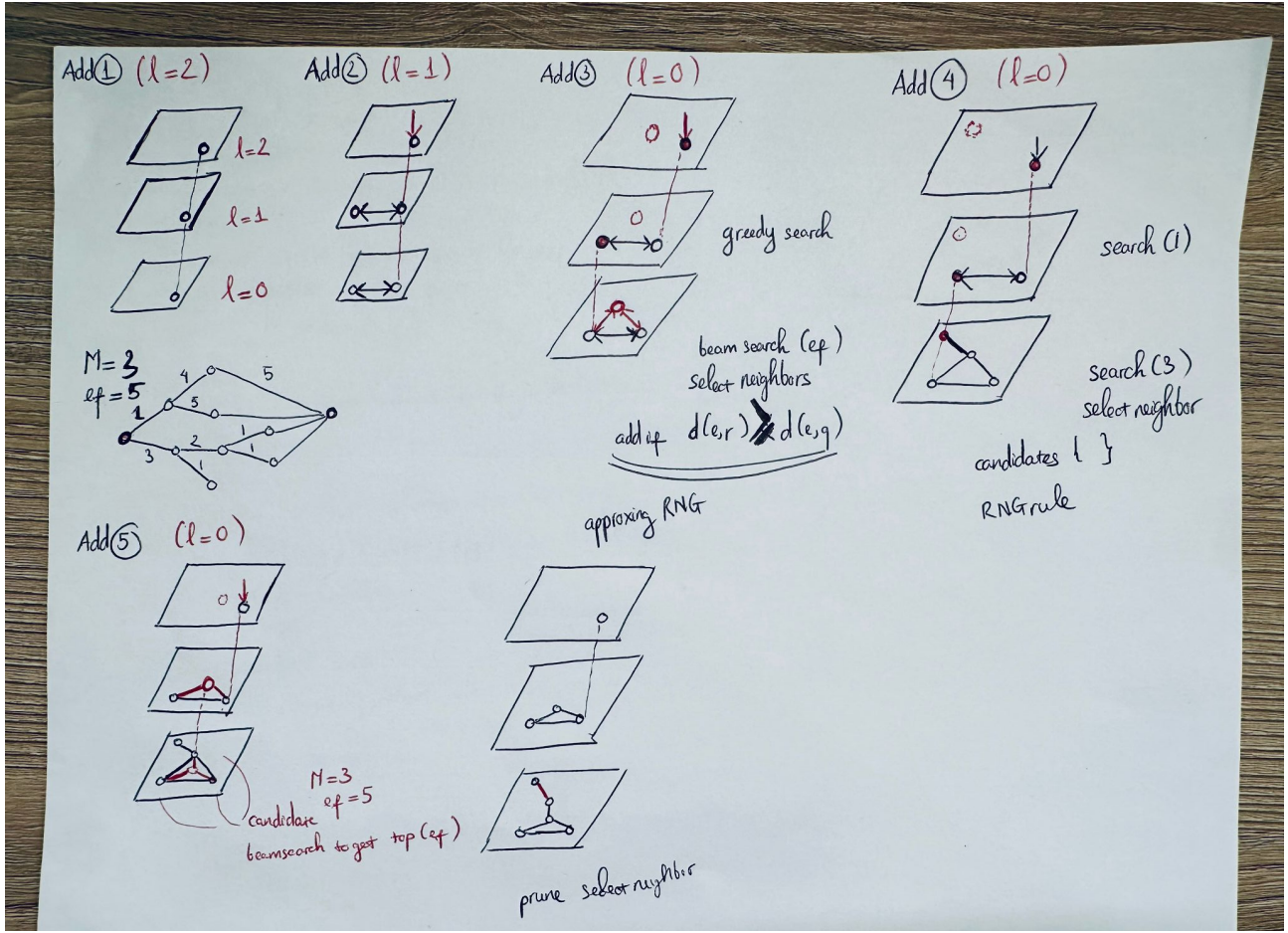


Figure 7: Routing procedure on a Relative Neighborhood Graph (RNG).

Example with 5 nodes. M (max number of outgoing edges = 3), efConstruction(5)



Indexing 1M 256D vectors with HNSW vs IVF

testing HNSW Flat

efSearch 16	0.011 ms per query, R@1 0.8740
efSearch 32	0.020 ms per query, R@1 0.9492
efSearch 64	0.033 ms per query, R@1 0.9779
efSearch 128	0.059 ms per query, R@1 0.9887
efSearch 256	0.104 ms per query, R@1 0.9920

testing IVF Flat (baseline)

nprobe 1	0.076 ms per query, R@1 0.4085
nprobe 4	0.067 ms per query, R@1 0.6331
nprobe 16	0.078 ms per query, R@1 0.8263
nprobe 64	0.141 ms per query, R@1 0.9470
nprobe 256	0.344 ms per query, R@1 0.9861

https://github.com/facebookresearch/faiss/wiki/Indexing-1M-vectors (2022)

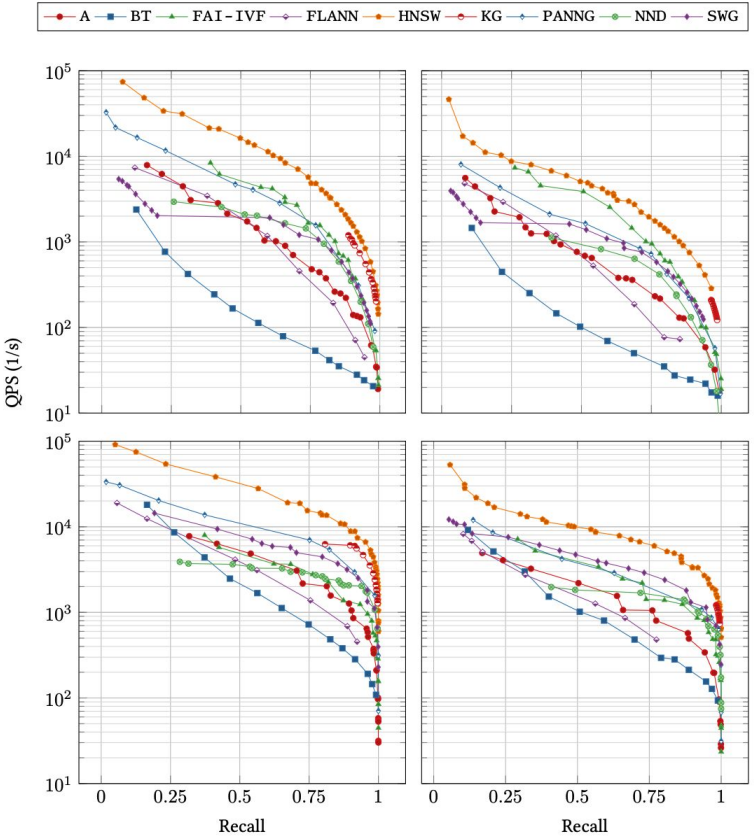


Figure 4 Recall-QPS (1/s) tradeoff - up and to the right is better. Top: GLOVE, bottom: SIFT; left: 10-NN, right: 100-NN.

HNSW Summary

- Fast, high-recall search with support for incremental insertions
- Uses a lot of memory, but if memory isn’t a constraint, HNSW is the preferred choice for Approximate Nearest Neighbor Search